Pipeline and roll-ups: `engine/build.py`

Runs the whole engine in order and writes the JSON files the dashboard reads (`public/data/`). It also computes the fleet roll-ups, daily cost, bill history, carbon and the summary numbers.

1. What it does

Inputs: `inventory.json` and `metrics.npz` from `generate.py` (simulated data) or from `csv_loader.py` (a company's own CSVs, via `--data <folder>`). **Outputs (in `public/data/`):** `summary`, `resources`, `recommendations`, `alerts`, `forecast`, `fleet`, `fleet_cells`, `assumptions`, and one `series/<vm id>.json` per VM.

2. Order of steps (`main`)

Step	What happens	Detail document
1	Load inventory and metrics	
2	Statistics for every VM	`STATS.md`
3	Monthly cost per resource and per-VM chart series	section 3
4	Per VM: not enough data? anomaly? else recommend	`RECOMMENDATION_ENGINE.md`, `ALERTS.md`
5	Orphan volumes and IPs	`RECOMMENDATION_ENGINE.md`
6	Short-window rule (under 30 days)	section 6
7	Number the cards, add carbon, mark defaults, add risk scores	section 5
8	Daily cost, current and projected month	section 4
9	Alerts	`ALERTS.md`
10	Cost forecast, usage forecast, backtest	`FORECAST.md`
11	Fleet roll-ups, bill history, summary, tiers	sections 7 to 9
12	Write all JSON files and print a summary	

3. Cost per resource and chart series

- **Monthly cost** = plan price + disk GB × ₹6 + network + IP price (₹300 for an IP). Network = the egress GB seen, scaled from the days of data the VM actually has to a 30.4-day month: `egress GB × ₹0.80 × 30.4167 ÷ days of data`.
- **Chart series:** each VM's 5-minute readings are reduced to 30-minute buckets (6 readings). Charts use the **average** per bucket, and CPU also keeps the **maximum** per

bucket so short peaks stay visible.

4. Daily cost and the monthly figures

For each of the 30 days, add up every resource that existed that day:

- VM: plan price ÷ 30.4167 + disk GB × ₹6 ÷ 30.4167 + that day's outbound GB × ₹0.80
- Volume: GB × ₹6 ÷ 30.4167
- IP: ₹300 ÷ 30.4167

Then:

- **Current monthly bill** = sum of every resource's monthly cost.
- **Projected next month** = the last day's cost × 30.4167. This is a run-rate: it includes the 8 ML VMs for a full month and the anomaly's extra egress.

5. Final touches on every recommendation

Field	Rule
id	`rec-001`, `rec-002`, … in creation order
saving	rounded to whole rupees
default_accepted	true unless the action is `review`, `keep` or `investigate`
carbon	`kWh = vCPUs removed × 4 W × 730 h × PUE 1.4 ÷ 1000`; `kg CO₂ = kWh × 0.716` (India grid); a tree absorbs about 21 kg CO_2 per year
risk_score	added by `attach_risk_score` (see the recommendation document)

6. Special cases handled in the pipeline

Case	What happens
More than 20% of readings missing, or under 2 days of data	"Not enough data" review card (HIGH risk, confidence 20). Its cost still counts
Active anomaly	"Sizing paused: investigate" card (HIGH risk, confidence 30)
VM reports less often than every 15 minutes (uploaded data)	A resize becomes a "review" card, because a short peak could hide between readings
Data window under 30 days	Resize, schedule and stop cards lose 10 points of confidence and move one risk step up (LOW → MEDIUM → HIGH)

7. Fleet roll-ups

- `fleet_rollup` (hourly): for each hour, `CPU % = Σ (VM CPU % × vCPU) ÷ Σ vCPU of VMs that existed`. The same for memory (weighted by RAM GB) and disk (weighted by disk GB). Network and IOPS are summed. This is **capacity weighted**: a 16-vCPU VM counts more than a 2-vCPU VM.

- **By region, team and environment:** the same weighting using each VM's average CPU, memory and current disk figures, plus total egress.
- `fleet_cells`: hourly sums for every region × environment × team combination that exists. Because the sums add up, the dashboard can filter by any mix of filters and still compute exact weighted utilisation in the browser.

8. Bill history

For the last 7 calendar months, `bill(month) = Σ over resources of (monthly cost × days the resource existed in that month ÷ days in the month)`. A resource's launch date (`created_at`) decides when it starts being billed. Nothing is assumed about growth: the curve is whatever the launch dates produce. It uses today's monthly cost for the whole history.

9. Summary numbers

- `current_monthly`, `projected_next_month`, `budget_monthly`
- `potential_saving` = the sum of savings of cards that are accepted by default and have a positive saving (so it **does not subtract** the cost of required upsizes)
- Cost split by environment, region, team, resource type and cost component (compute, storage, network, IP)
- `target_tiers` (see `TARGET_PLANNER.md`) and the count of each action type

Sample (my run, seed 42): current ₹9,30,948 per month; projected ₹9,66,581; potential saving ₹2,12,877 (23%). The four upsizes cost about ₹12,000, so the net saving if everything default is accepted is about ₹2,00,877.

10. Assumptions

- Every resource costs the same in every region.
- Past launch dates and today's price are enough to rebuild the monthly bill history.
- Egress seen in the window is a fair guide to a month.
- A 30.4167-day month (730 ÷ 24 hours).

11. Limitations

1. **Potential saving is gross.** It leaves out the upsize cost; say "net" only if you subtract it.
2. **Run-rate projection.** One day's cost times 30.4 reacts strongly to a single unusual day (such as the day of the anomaly's heavy egress).
3. **History is reconstructed, not measured.** It applies today's cost to past months from launch dates.
4. **Sequential rules.** The first matching rule decides. A VM with an anomaly never gets any sizing card.
5. **One run, one dataset.** Re-running replaces the previous JSON files and the old per-VM series files.